

Designing and Deploying Scalable Microservices: From Architecture to DevOps

Comprehensive Q&A Study Guide

with Diagrams, Answers & Detailed Explanations – By Sasuni Wljerathne

Topics Covered

Section	Topic	Questions	Diagrams
Section 1	Architectural Patterns	8 (Q1–Q8)	1 diagram
Section 2	Design Patterns & SOLID Principles	8 (Q9–Q16)	1 mind map
Section 3	DevOps & Virtualization	8 (Q17–Q24)	2 diagrams
Section 4	Networking & Web Technologies	8 (Q25–Q32)	1 diagram
Section 5	Version Control — Git	6 (Q33–Q38)	1 diagram
Section 6	Programming Ecosystems	7 (Q39–Q45)	1 diagram

SECTION 1: Architectural Patterns

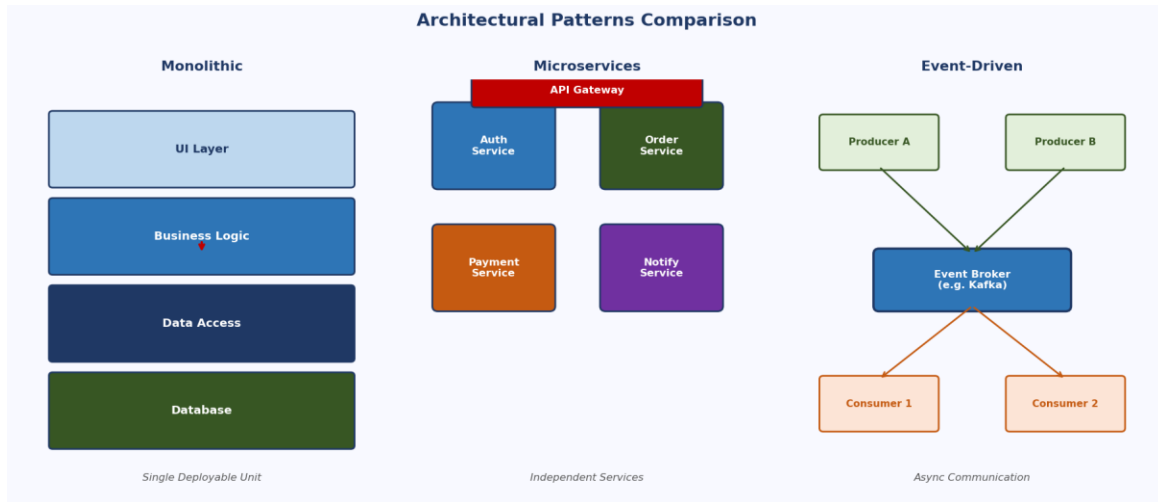


Figure 1: Monolithic vs Microservices vs Event-Driven Architecture

Q1. Which architectural pattern is best described as having "dumb pipes and smart endpoints"?

- A) Service-Oriented Architecture (SOA)
- B) Microservices Architecture ✓ CORRECT**
- C) Monolithic Architecture
- D) Space-Based Architecture

Answer: B | Microservices prefer simple communication channels (HTTP, messaging) and put logic in the services (endpoints). SOA, by contrast, uses smart ESB (Enterprise Service Bus) pipes that handle routing, transformation, and orchestration.

Q2. In a 3-Tier architecture, which tier is responsible for ONLY storing and retrieving data?

- A) Presentation Tier
- B) Application/Logic Tier
- C) Data Tier ✓ CORRECT**
- D) Integration Tier

Answer: C | 3-Tier Architecture: Tier 1 = Presentation (UI), Tier 2 = Business Logic (application server), Tier 3 = Data (database). The Data Tier has no business logic — it stores, retrieves, and manages raw data.

Q3. The CAP theorem requires distributed systems to choose two of three guarantees during a network partition. Which two does Amazon DynamoDB (AP system) prioritize?

- A) Consistency and Partition Tolerance
- B) Availability and Partition Tolerance ✓ CORRECT**
- C) Consistency and Availability
- D) Consistency and Durability

Answer: B | DynamoDB is an AP system: it prioritizes Availability (always responds) and Partition Tolerance (works despite network splits), at the cost of Consistency (data may be briefly stale). CP systems (MongoDB) prioritize Consistency over Availability.

Q4. Which architectural pattern uses "processing units" each with its own in-memory copy of data, eliminating the shared database bottleneck?

- A) Microservices
- B) Layered Architecture
- C) Space-Based Architecture (SBA) ✓ CORRECT**
- D) Client-Server

Answer: C | SBA (also called Tuple Space Architecture) distributes both data AND processing across in-memory grids. Each Processing Unit has its own data copy. A Data Replication Engine synchronizes state. Used for extreme scalability like financial trading platforms.

Q5. What is "Back Pressure" in reactive and event-driven systems?

- A) The force applied when rolling back a failed deployment
- B) A mechanism where consumers signal producers to slow down when overwhelmed ✓ CORRECT**
- C) A load balancing algorithm that redirects traffic to less busy servers
- D) A type of caching strategy that pre-loads data before it is requested

Answer: B | Back pressure prevents a fast producer from overwhelming a slow consumer. In reactive systems (RxJava, Project Reactor), consumers signal capacity. Without it, unbounded queues grow until memory runs out — a common microservices failure mode.

Q6. In Service-Oriented Architecture (SOA), an Enterprise Service Bus (ESB) primarily handles:

- A) Containerizing services for deployment
- B) Message routing, transformation, and orchestration between services ✓ CORRECT**
- C) Providing a shared database for all services
- D) Authenticating users across all service calls

Answer: B | The ESB is the central backbone in SOA — it routes messages, transforms data formats (XML to JSON), applies business rules, handles protocol mediation, and orchestrates service interactions. Its centralization is both its strength and its weakness (single point of failure).

Q7. Which pattern describes a system where all state is stored as a sequence of immutable events, and current state is derived by replaying them?

- A) CQRS (Command Query Responsibility Segregation)
- B) Event Sourcing ✓ CORRECT**
- C) Saga Pattern
- D) Circuit Breaker Pattern

Answer: B | Event Sourcing stores every state change as an immutable event (e.g., "OrderPlaced", "PaymentProcessed"). Current state is rebuilt by replaying events. Provides full audit trail, temporal queries, and easy debugging — used in banking and e-commerce.

Q8. The Circuit Breaker pattern in microservices is analogous to:

- A) A database transaction that rolls back on failure
- B) An electrical circuit breaker that stops current flow when overloaded ✓ CORRECT**
- C) A load balancer that routes traffic evenly
- D) A retry mechanism that attempts failed requests indefinitely

Answer: B | Just like an electrical breaker, the Circuit Breaker pattern "opens" (stops calls) when a service fails repeatedly — preventing cascading failures. After a timeout, it "half-opens" to test recovery. States: Closed (normal) -> Open (failing) -> Half-Open (testing).

SECTION 2: Design Patterns & Principles

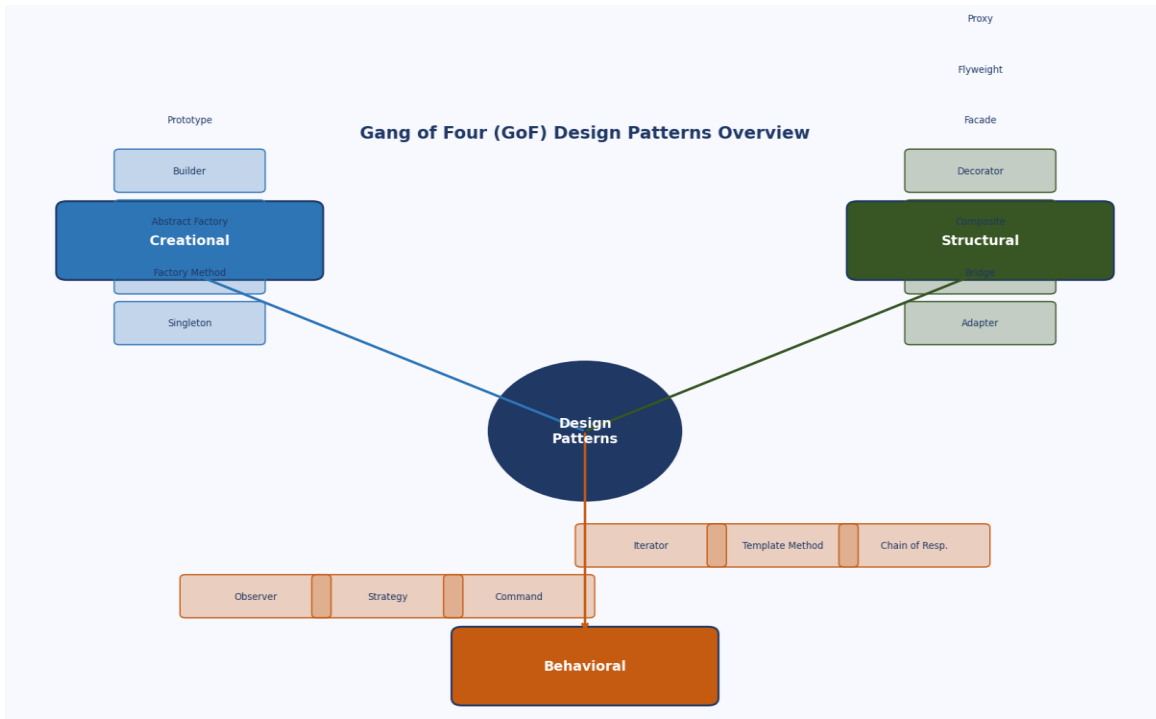


Figure 2: Gang of Four (GoF) Design Patterns — Creational, Structural & Behavioral

Q9. The "Tell, Don't Ask" principle means you should:

- A) Always use verbose variable names instead of single letters
- B) Command objects to perform actions rather than querying their state and deciding externally** ✓ CORRECT
- C) Avoid using questions (if-statements) in your code
- D) Use event-driven patterns instead of synchronous calls

Answer: B | Tell, Don't Ask: instead of asking an object for its data and making decisions about it externally, tell the object what to do. Example: instead of "if account.getBalance() > amount" externally, call "account.withdraw(amount)" which handles the logic internally.

Q10. The "Law of Demeter" (Principle of Least Knowledge) states:

- A) A method should only call methods on: itself, its parameters, objects it creates, or its direct components** ✓ CORRECT
- B) Classes should only have a maximum of 10 methods
- C) All dependencies must be injected through constructors
- D) Public methods should never access private fields directly

Answer: A | The Law of Demeter prevents "train wreck" code: `customer.getOrder().getItem().getPrice()`. Each dot is a dependency on internal structure. The law says: talk only to your immediate friends, not friends of friends.

Q11. Which design pattern is used to represent an operation to be performed on elements of an object structure, allowing new operations without modifying those elements?

- A) Iterator
- B) Composite
- C) Visitor** ✓ CORRECT
- D) State

Answer: C | The Visitor pattern separates algorithms from the objects they operate on. You add a new Visit operation without changing the element classes. Classic use: AST (Abstract Syntax Tree) processing in compilers — add new analysis passes without touching the AST nodes.

Q12. What is the primary difference between the Factory Method and Abstract Factory patterns?

- A) Factory Method creates families of objects; Abstract Factory creates one type
- B) Abstract Factory creates families of related objects; Factory Method creates one product type via subclassing ✓ CORRECT**
- C) They are identical patterns with different names
- D) Factory Method uses interfaces; Abstract Factory uses abstract classes only

Answer: B | Factory Method: one product, one creator (subclass decides what to instantiate). Abstract Factory: FAMILIES of related products (e.g., UI widgets: WindowsButton+WindowsCheckbox vs MacButton+MacCheckbox). Think of Abstract Factory as a "theme" factory.

Q13. The Memento behavioral pattern is primarily used to:

- A) Share state between multiple objects efficiently
- B) Capture and restore an object's internal state for undo/rollback operations ✓ CORRECT**
- C) Define a one-to-many dependency so state changes are broadcast
- D) Provide an interface for traversing a collection

Answer: B | Memento captures an object's state into a "memento" object without violating encapsulation. The Caretaker stores mementos for undo operations. Used in text editors (Ctrl+Z), game save states, and transaction management.

Q14. YAGNI stands for "You Aren't Gonna Need It." It advises developers to:

- A) Avoid writing documentation until features are finalized
- B) Not implement functionality until it is actually needed ✓ CORRECT**
- C) Never refactor working code
- D) Skip unit testing for simple utility functions

Answer: B | YAGNI (XP principle by Ron Jeffries) prevents over-engineering. Building features "just in case" wastes time and adds complexity. Work on what the current requirements actually need. Pair with KISS (Keep It Simple, Stupid) for clean design.

Q15. The Null Object Pattern solves which common problem?

- A) Preventing null pointer exceptions by providing a default, do-nothing object instead of null ✓ CORRECT**
- B) Optimizing database queries that return empty result sets
- C) Handling division by zero in mathematical operations
- D) Managing empty collections in recursive algorithms

Answer: A | Instead of returning null (requiring null checks everywhere), you return a "null object" that implements the interface with do-nothing or default behavior. Example: a NullLogger that implements Logger but does nothing, eliminating "if (logger != null) logger.log(msg)."

Q16. Which SOLID principle is MOST directly violated when a class has 15 public methods handling authentication, email sending, database operations, and report generation?

- A) Open/Closed Principle
- B) Liskov Substitution Principle
- C) Single Responsibility Principle ✓ CORRECT**
- D) Interface Segregation Principle

Answer: C | A class doing authentication + email + database + reporting has at least 4 reasons to change — a clear SRP violation. Split into AuthService, EmailService, UserRepository, and ReportService. Each has one clear responsibility.

SECTION 3: DevOps & Virtualization

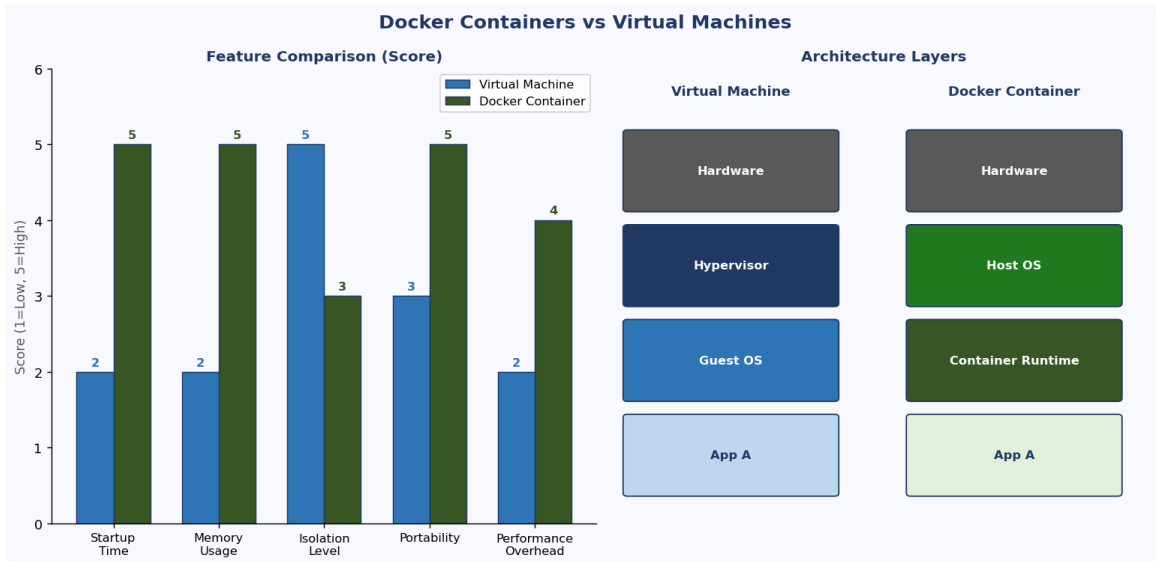


Figure 3: Docker Containers vs Virtual Machines — Feature Comparison & Architecture Layers

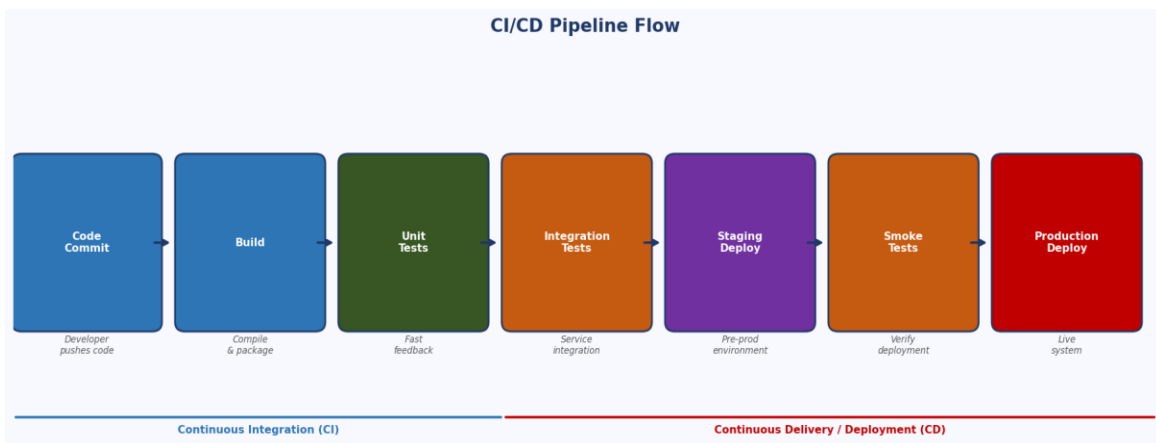


Figure 4: CI/CD Pipeline — From Code Commit to Production Deployment

Q17. In Docker, what is a "layer" in the context of image building?

- A) A separate Docker container that provides a service
- B) A read-only filesystem change created by each Dockerfile instruction, cached for reuse ✓ CORRECT**
- C) A network segment that isolates containers
- D) A volume mount point for persistent storage

Answer: B | Each Dockerfile instruction (RUN, COPY, ADD) creates an immutable layer. Layers are cached — if an instruction's context hasn't changed, Docker reuses the cached layer. This makes rebuilds fast. Order matters: put frequently changing instructions (COPY code) AFTER rarely changing ones (RUN apt-get install).

Q18. What is "container orchestration" and why is it needed?

- A) The process of building Docker images from Dockerfiles
- B) Automated management of containerized applications across multiple hosts, handling scaling, failover, and networking ✓ CORRECT**
- C) A method of writing Docker Compose files for local development
- D) A technique for optimizing container startup time

Answer: B | A single Docker host can't handle production load. Orchestration (Kubernetes, Docker Swarm) manages: deploying containers across a cluster, auto-scaling, health checks/restarts, service discovery, load balancing, rolling updates, and secret management.

Q19. In Kubernetes, what is the difference between a "Deployment" and a "StatefulSet"?

- A) Deployments are for databases; StatefulSets are for web applications
- B) Deployments manage stateless Pods with interchangeable replicas; StatefulSets manage stateful Pods with stable network identities and persistent storage ✓ CORRECT**
- C) StatefulSets cannot be scaled; Deployments can
- D) They are identical — StatefulSet is the newer name for Deployment

Answer: B | Deployment: pods are identical and interchangeable (web servers). StatefulSet: each Pod has a stable hostname (pod-0, pod-1), stable storage, and ordered deployment/deletion — critical for databases (MySQL, Cassandra) where node identity matters.

Q20. What does "immutable infrastructure" mean in DevOps?

- A) Servers that cannot be scaled horizontally
- B) Servers are never modified after deployment; instead they are replaced with new versions ✓ CORRECT**
- C) Infrastructure configuration files that are write-protected
- D) A Kubernetes policy preventing container restarts

Answer: B | Immutable Infrastructure: instead of patching running servers (mutable), you build a new image, test it, and replace the old server entirely. Eliminates configuration drift, makes rollbacks trivial (redploy old image), and ensures environments are reproducible.

Q21. What is the purpose of a Kubernetes "Ingress" resource?

- A) To define environment variables for Pods
- B) To manage external access to services, providing HTTP routing, SSL termination, and virtual hosting ✓ CORRECT**
- C) To allocate persistent storage volumes to Pods
- D) To enforce network policies between Pods in different namespaces

Answer: B | Ingress manages external HTTP/HTTPS traffic into the cluster. An Ingress controller (NGINX, Traefik) routes requests based on hostname (api.example.com) or path (/api, /admin) to different Services. It also handles SSL termination — so your Pods don't need to deal with TLS.

Q22. What is "blue-green deployment"?

- A) A deployment strategy using two identical environments, switching traffic from one to the other for zero-downtime releases ✓ CORRECT**
- B) A branching strategy where blue branches are features and green branches are hotfixes
- C) A Kubernetes deployment using blue pods for CPU-intensive and green pods for memory-intensive tasks
- D) A monitoring strategy that uses two different alert systems

Answer: A | Blue = current live. Green = new version deployed alongside. After testing green, switch the load balancer to send all traffic to green. Blue stays idle for instant rollback. Zero downtime, but requires double infrastructure. Contrast with Rolling Updates (gradual) and Canary (partial traffic).

Q23. In a Dockerfile, what is the difference between CMD and ENTRYPOINT?

- A) CMD runs during build; ENTRYPOINT runs at container start
- B) ENTRYPOINT defines the executable that always runs; CMD provides default arguments that can be overridden ✓ CORRECT**
- C) They are identical — both define the startup command
- D) CMD is for Linux containers; ENTRYPOINT is for Windows containers

Answer: B | *ENTRYPOINT: fixed executable (e.g., ENTRYPOINT ["python3"]). CMD: default arguments (e.g., CMD ["app.py"]). Together: container runs "python3 app.py". User can override CMD at runtime (docker run myimage script.py) but not ENTRYPOINT (without --entrypoint flag).*

Q24. What is "canary deployment" and what problem does it solve?

- A) Deploying only to servers named after canary birds for testing
- B) Gradually rolling out a new version to a small percentage of users first, limiting blast radius if issues arise ✓ CORRECT**
- C) A deployment strategy that requires manual approval at every stage
- D) A method of deploying to production only during low-traffic hours

Answer: B | *Canary (named after canaries in coal mines): route 5% of traffic to new version, monitor error rates/latency, then gradually increase to 100% if healthy. Limits impact of bugs to a small user subset. If issues appear, route all traffic back instantly. Used by Netflix, Google, and Facebook.*

SECTION 4: Networking & Web Technologies

REST API - HTTP Methods Quick Reference				
Method	Example URI	Operation	Idempotent?	Safe?
GET	/users	Read / Retrieve	Yes	Yes
POST	/users	Create new resource	No	No
PUT	/users/1	Full Update (replace)	Yes	No
PATCH	/users/1	Partial Update	No	No
DELETE	/users/1	Delete resource	Yes	No

Figure 5: REST API HTTP Methods — Operations, Idempotency & Safety Reference

Q25. What is the difference between HTTP/1.1 and HTTP/2?

- A) HTTP/2 uses UDP instead of TCP
- B) HTTP/2 supports multiplexing (multiple requests over one connection), header compression, and server push ✓ CORRECT**
- C) HTTP/1.1 supports TLS; HTTP/2 does not
- D) HTTP/2 is only for internal APIs, not public-facing web pages

Answer: B | HTTP/1.1 opens a new TCP connection per request (or uses keep-alive with head-of-line blocking). HTTP/2 multiplexes multiple requests/responses over ONE TCP connection simultaneously, compresses headers (HPACK), and allows server push (sending resources before the client asks). Result: faster page loads.

Q26. What is the purpose of an SSL/TLS certificate's "Common Name" (CN) or "Subject Alternative Name" (SAN)?

- A) To store the certificate owner's private key
- B) To specify the domain name(s) the certificate is valid for ✓ CORRECT**
- C) To define the encryption algorithm used for the connection
- D) To identify the certificate authority's server address

Answer: B | The CN (legacy) or SAN (modern, allows multiple domains) tells the browser which hostnames this certificate is valid for. When you connect to api.example.com, the browser checks if the certificate's SAN includes api.example.com. Mismatch = security warning.

Q27. What is DNS poisoning (cache poisoning)?

- A) Overloading a DNS server with too many queries
- B) Inserting false DNS records into a resolver's cache to redirect users to malicious servers ✓ CORRECT**
- C) Encrypting DNS queries to prevent eavesdropping
- D) Blocking DNS queries from unauthorized clients

Answer: B | DNS Cache Poisoning: an attacker injects forged DNS responses into a resolver's cache. When users query "bank.com," the poisoned cache returns the attacker's IP. DNSSEC (DNS Security Extensions) mitigates this by digitally signing DNS records.

Q28. What is the key difference between SQL injection and NoSQL injection?

- A) SQL injection is only possible in web applications; NoSQL injection only in mobile apps
- B) SQL injection manipulates structured query syntax; NoSQL injection exploits query operators (e.g., MongoDB \$where, \$gt) ✓ CORRECT**

- C) NoSQL injection is impossible because NoSQL databases have no query language
- D) They are identical attacks with different names

Answer: B | SQL injection: manipulates SQL syntax ('OR 1=1). NoSQL injection (e.g., MongoDB): exploits JSON operators — sending {password: {\$gt: ""}} bypasses password checks since \$gt (greater than) matches any non-empty string. Always sanitize inputs and use parameterized queries regardless of database type.

Q29. What does "idempotency" mean for a REST API endpoint, and why does it matter?

- A) The endpoint processes requests in parallel for performance
- B) Making the same request multiple times produces the same result as making it once, enabling safe retries ✓ CORRECT**
- C) The endpoint returns the same data format regardless of input type
- D) The endpoint caches all responses to reduce server load

Answer: B | Idempotency enables safe retries on network failure. If a DELETE /users/1 request times out, you can safely retry it — the user is already deleted, no harm done. POST is NOT idempotent (retrying creates duplicates). This is why payment APIs use idempotency keys.

Q30. In NGINX, what is the purpose of the "upstream" block combined with "proxy_pass"?

- A) To serve static files from a local directory efficiently
- B) To define a pool of backend servers and forward client requests to them (reverse proxy/load balancing) ✓ CORRECT**
- C) To configure SSL certificates for HTTPS termination
- D) To set rate limits on incoming client connections

Answer: B | Example: upstream backend { server app1:8080; server app2:8080; } and proxy_pass http://backend; — NGINX distributes requests across both app servers using round-robin by default. Add "least_conn;" for least-connections load balancing, or "ip_hash;" for session persistence.

Q31. What is "OAuth 2.0" and how does it differ from "OpenID Connect" (OIDC)?

- A) OAuth 2.0 handles authentication; OIDC handles authorization
- B) OAuth 2.0 is an authorization framework for resource access delegation; OIDC adds an identity layer on top for authentication ✓ CORRECT**
- C) They are competing standards that cannot be used together
- D) OAuth 2.0 is for mobile apps; OIDC is for web applications only

Answer: B | OAuth 2.0: authorization — "Can this app access my Google Drive?" Returns an access_token. OIDC: authentication built on OAuth 2.0 — "Who is this user?" Returns an id_token (JWT) with user identity claims. Google Sign-In, "Login with GitHub" use OIDC.

Q32. What is a "CDN" (Content Delivery Network) and what problem does it solve?

- A) A type of database distributed across multiple regions for low-latency reads
- B) A geographically distributed network of servers that cache and serve static content from locations closer to users ✓ CORRECT**
- C) A cloud service that compiles and bundles JavaScript files
- D) A network monitoring tool that tracks API response times globally

Answer: B | A CDN (Cloudflare, Akamai, AWS CloudFront) caches static assets (images, JS, CSS, videos) at edge nodes worldwide. A user in Sri Lanka gets content from a nearby Singapore node instead of a US origin server — reducing latency from 200ms to 20ms. Also mitigates DDoS attacks.

SECTION 5: Version Control — Git

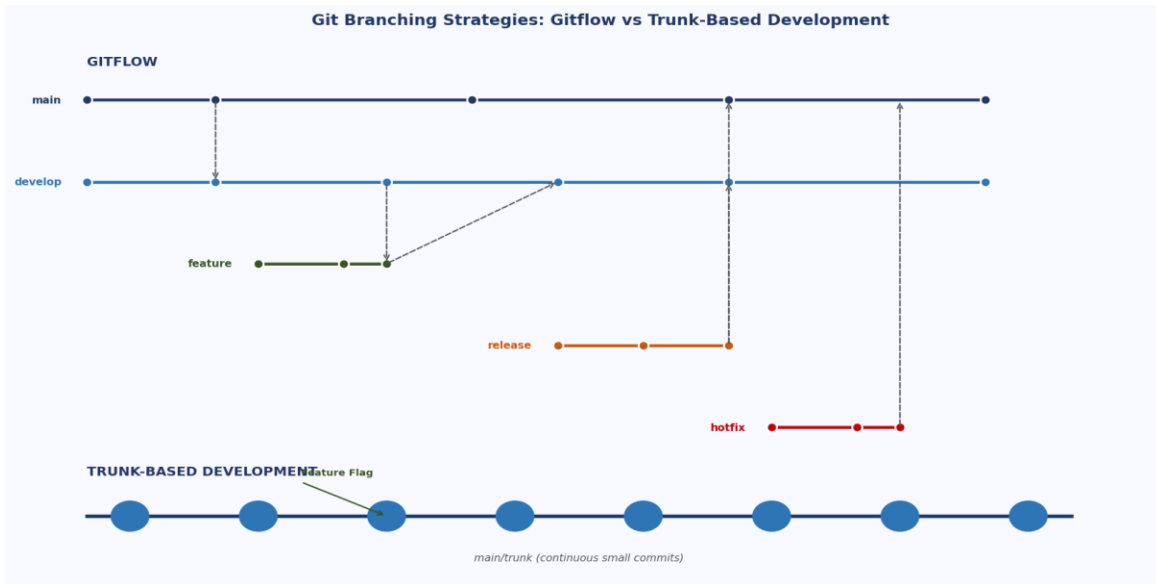


Figure 6: Git Branching Strategies — Gitflow (top) vs Trunk-Based Development (bottom)

Q33. What is the purpose of "git reflog" and when would you use it?

- A) To display the formatted commit log with graph visualization
- B) To recover lost commits or branches by showing every HEAD movement, even after reset or rebase ✓ CORRECT**
- C) To view the remote repository's commit history
- D) To compare two branches and show diverging commits

Answer: B | *git reflog records every change to HEAD (commits, resets, checkouts). If you accidentally do "git reset --hard HEAD~5" or lose a branch, reflog lets you find the lost commit SHA and recover it with "git checkout -b recovery <sha>". It is your safety net for Git mistakes.*

Q34. What does "git reset --soft HEAD~1" do?

- A) Permanently deletes the last commit and all its changes
- B) Undoes the last commit but keeps all changes staged (in the index) ✓ CORRECT**
- C) Moves the last commit to a new branch
- D) Reverts the last commit by creating a new inverse commit

Answer: B | *Three reset modes: --soft (undo commit, keep staged), --mixed (undo commit, unstage changes, keep in working dir), --hard (undo commit, DISCARD all changes permanently). Use --soft to "uncommit" before amending and recommitting.*

Q35. What is a "Git hook" and how is it used in practice?

- A) A special branch that is protected from direct commits
- B) A script that automatically runs at specific Git lifecycle events like pre-commit, post-merge, or pre-push ✓ CORRECT**
- C) A feature that links two repositories together
- D) A configuration option that prevents force-pushing to main

Answer: B | *Git hooks are scripts in .git/hooks/ that trigger on Git events. pre-commit: run linting/tests before committing. commit-msg: enforce commit message format. pre-push: run full test suite before pushing. Tools like Husky make hooks easy to configure and share in npm projects.*

Q36. What is the "Squash and Merge" strategy in pull requests?

- A) Merging a branch without creating any commit on the target branch

B) Combining all commits from a feature branch into a single commit before merging, keeping the target branch history clean ✓ CORRECT

C) Rewriting the entire target branch history to remove merge commits

D) A conflict resolution strategy that accepts all incoming changes

Answer: B | "Squash and Merge" collapses 20 messy WIP commits ("fix typo", "debugging", "finally works") from a feature branch into ONE clean commit on main. Great for clean history but loses granular commit context. GitHub, GitLab, and Bitbucket all offer this as a merge strategy option.

Q37. In Git, what does "git blame" show you?

A) A list of all bugs introduced by each developer

B) The last commit and author that modified each line of a file ✓ CORRECT

C) All uncommitted changes that caused test failures

D) A diff between the working directory and the last commit

Answer: B | *git blame* shows every line of a file annotated with: commit hash, author name, date, and line number. Used to find who last modified a specific line — critical for understanding context or finding who introduced a bug. Not for blame/shame, but for context.

Q38. What is "Semantic Versioning" (SemVer) and what do the three numbers mean?

A) A method of naming Git branches based on JIRA ticket numbers

B) A versioning scheme (MAJOR.MINOR.PATCH) where MAJOR=breaking changes, MINOR=new features, PATCH=bug fixes ✓ CORRECT

C) A commit message format specifying the type, scope, and description of changes

D) A tag naming convention where numbers represent release dates

Answer: B | *SemVer (v2.3.1): MAJOR (2) = incompatible API changes (breaking). MINOR (3) = backward-compatible new features. PATCH (1) = backward-compatible bug fixes. Given v1.4.2: v2.0.0 = breaking change, v1.5.0 = new feature, v1.4.3 = bug fix. Used by npm, Maven, pip, and most package managers.*

SECTION 6: Programming Ecosystems

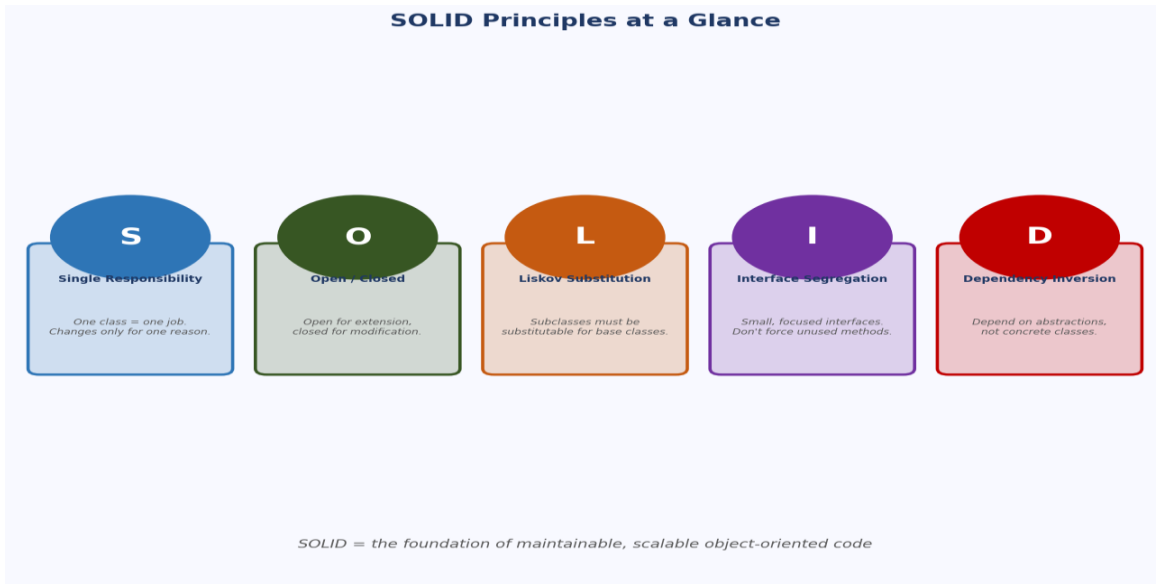


Figure 7: SOLID Principles Summary — The Foundation of Clean Object-Oriented Design

Q39. What is "garbage collection" in programming languages, and what is its trade-off?

- A) A tool that removes unused imports and dead code from source files
- B) Automatic memory management that frees unused objects, trading programmer convenience for potential pauses ✓ CORRECT**
- C) A process that cleans up temporary files created during compilation
- D) A database maintenance operation that removes orphaned records

Answer: B | GC (Java, Go, Python, C#) automatically reclaims memory of unreachable objects, preventing memory leaks. Trade-off: GC pauses (Stop-The-World in Java) can cause latency spikes. Languages like C and Rust give manual control for predictable performance. Java's G1 and ZGC minimize pauses.

Q40. What is "reflection" in programming, and when is it used?

- A) The process of copying an object's state to create a duplicate
- B) The ability of a program to inspect and modify its own structure and behavior at runtime ✓ CORRECT**
- C) A design pattern for implementing bidirectional data binding
- D) A compiler optimization technique that inlines function calls

Answer: B | Reflection allows runtime inspection of classes, methods, and fields without knowing them at compile time. Used by: Java serialization, Spring dependency injection (scanning @Autowired), JUnit (discovering @Test methods), ORM frameworks (mapping fields to DB columns). Powerful but slow — avoid in performance-critical paths.

Q41. What is the difference between "concurrency" and "parallelism"?

- A) They are synonymous terms for simultaneous execution
- B) Concurrency is dealing with multiple tasks at once (interleaving on one core); parallelism is actually executing multiple tasks simultaneously (multiple cores) ✓ CORRECT**
- C) Concurrency uses multiple processes; parallelism uses multiple threads
- D) Parallelism is for I/O-bound tasks; concurrency is for CPU-bound tasks

Answer: B | Concurrency: structure — managing multiple tasks that may overlap in time (a waiter serving many tables). Parallelism: execution — multiple tasks running truly simultaneously (multiple waiters). A single-core CPU can be concurrent (task switching) but not truly parallel. Node.js achieves concurrency (async I/O) on a single thread.

Q42. What is a "monad" in functional programming?

- A) A single-method interface in Java that enables lambda expressions
- B) A design pattern/structure for chaining computations that handle context (like null, errors, or async) in a composable way ✓ CORRECT**
- C) A type of recursive data structure used in tree traversal
- D) A pure function with no side effects

Answer: B | A monad wraps a value with context and provides flatMap (or bind) for chaining operations. Examples: Optional/Maybe (null handling), Promise/Future (async), Result/Either (error handling). Java Optional, JavaScript Promise, Haskell Maybe are all monadic. They compose operations without explicit null checks or try-catches at every step.

Q43. What is the difference between "compiled" languages and "transpiled" languages?

- A) Compiled languages produce machine code; transpiled languages convert source code to another high-level language ✓ CORRECT**
- B) Transpiled languages are always faster than compiled ones
- C) Compiled languages support OOP; transpiled languages only support functional programming
- D) Transpiled languages run in the browser; compiled languages run on servers

Answer: A | Compiled: source -> machine code (C, Go, Rust). Transpiled: source -> another source language (TypeScript -> JavaScript, SCSS -> CSS, CoffeeScript -> JavaScript, Kotlin -> JavaScript). TypeScript is the most prominent transpilation example — you write TS, the compiler outputs JS.

Q44. In Python, what is the Global Interpreter Lock (GIL) and its impact?

- A) A security feature that prevents Python scripts from accessing global variables
- B) A mutex that allows only one thread to execute Python bytecode at a time, limiting true multi-threaded CPU parallelism ✓ CORRECT**
- C) A memory optimization that caches global variables for faster access
- D) A compiler feature that optimizes global function calls

Answer: B | CPython's GIL prevents multiple threads from executing Python bytecode simultaneously — so CPU-bound multi-threaded Python code doesn't benefit from multiple cores. Workarounds: multiprocessing module (separate processes), async/await (for I/O-bound), or use PyPy/Cython. NumPy releases the GIL for C-level operations.

Q45. What is "tree shaking" in the context of JavaScript bundlers like Webpack or Rollup?

- A) A garbage collection algorithm that removes circular references in JavaScript objects
- B) A dead code elimination technique that removes unused exported code from the final bundle ✓ CORRECT**
- C) A process that restructures deeply nested DOM trees for better rendering performance
- D) A dependency analysis that detects conflicting package versions

Answer: B | Tree shaking statically analyzes ES6 import/export statements to identify and remove unused code. If you import { debounce } from 'lodash-es', only debounce is included — not all of lodash (70KB+). Requires ES modules (not CommonJS require()). Significantly reduces bundle size.

COMPLETE ANSWER KEY & QUICK REFERENCE

Answers at a Glance

Q	Ans	Q	Ans	Q	Ans	Q	Ans	Q	Ans
Q1	B	Q10	A	Q19	B	Q28	B	Q37	B
Q2	C	Q11	C	Q20	B	Q29	B	Q38	B
Q3	B	Q12	B	Q21	B	Q30	B	Q39	B
Q4	C	Q13	B	Q22	A	Q31	B	Q40	B
Q5	B	Q14	B	Q23	B	Q32	B	Q41	B
Q6	B	Q15	A	Q24	B	Q33	B	Q42	B
Q7	B	Q16	C	Q25	B	Q34	B	Q43	A
Q8	B	Q17	B	Q26	B	Q35	B	Q44	B
Q9	B	Q18	B	Q27	B	Q36	B	Q45	B

Section-by-Section Answer Summary

SECTION 1: Architectural Patterns

Question	Answer	Key Concept
Q1	B	Microservices prefer simple communication channels (HTTP, messaging) and put logic in the services (endpoints).
Q2	C	3-Tier Architecture: Tier 1 = Presentation (UI), Tier 2 = Business Logic (application server), Tier 3 = Data (database).
Q3	B	DynamoDB is an AP system: it prioritizes Availability (always responds) and Partition Tolerance (works despite network splits), at the cost of Consistency (data may be briefly stale).
Q4	C	SBA (also called Tuple Space Architecture) distributes both data AND processing across in-memory grids.
Q5	B	Back pressure prevents a fast producer from overwhelming a slow consumer.
Q6	B	The ESB is the central backbone in SOA — it routes messages, transforms data formats (XML to JSON), applies business rules, handles protocol mediation, and orchestrates service interactions.
Q7	B	Event Sourcing stores every state change as an immutable event (e.
Q8	B	Just like an electrical breaker, the Circuit Breaker pattern "opens" (stops calls) when a service fails repeatedly — preventing cascading failures.

SECTION 2: Design Patterns & Principles

Question	Answer	Key Concept
Q9	B	Tell, Don't Ask: instead of asking an object for its data and making decisions about it externally, tell the object what to do.
Q10	A	The Law of Demeter prevents "train wreck" code: customer.
Q11	C	The Visitor pattern separates algorithms from the objects they operate on.
Q12	B	Factory Method: one product, one creator (subclass decides what to instantiate).

Question	Answer	Key Concept
Q13	B	Memento captures an object's state into a "memento" object without violating encapsulation.
Q14	B	YAGNI (XP principle by Ron Jeffries) prevents over-engineering.
Q15	A	Instead of returning null (requiring null checks everywhere), you return a "null object" that implements the interface with do-nothing or default behavior.
Q16	C	A class doing authentication + email + database + reporting has at least 4 reasons to change — a clear SRP violation.

SECTION 3: DevOps & Virtualization

Question	Answer	Key Concept
Q17	B	Each Dockerfile instruction (RUN, COPY, ADD) creates an immutable layer.
Q18	B	A single Docker host can't handle production load.
Q19	B	Deployment: pods are identical and interchangeable (web servers).
Q20	B	Immutable Infrastructure: instead of patching running servers (mutable), you build a new image, test it, and replace the old server entirely.
Q21	B	Ingress manages external HTTP/HTTPS traffic into the cluster.
Q22	A	Blue = current live.
Q23	B	ENTRYPOINT: fixed executable (e.
Q24	B	Canary (named after canaries in coal mines): route 5% of traffic to new version, monitor error rates/latency, then gradually increase to 100% if healthy.

SECTION 4: Networking & Web Technologies

Question	Answer	Key Concept
Q25	B	HTTP/1.
Q26	B	The CN (legacy) or SAN (modern, allows multiple domains) tells the browser which hostnames this certificate is valid for.
Q27	B	DNS Cache Poisoning: an attacker injects forged DNS responses into a resolver's cache.
Q28	B	SQL injection: manipulates SQL syntax ('OR 1=1).
Q29	B	Idempotency enables safe retries on network failure.
Q30	B	Example: upstream backend { server app1:8080; server app2:8080; } and proxy_pass http://backend; — NGINX distributes requests across both app servers using round-robin by default.
Q31	B	OAuth 2.
Q32	B	A CDN (Cloudflare, Akamai, AWS CloudFront) caches static assets (images, JS, CSS, videos) at edge nodes worldwide.

SECTION 5: Version Control — Git

Question	Answer	Key Concept
Q33	B	git reflog records every change to HEAD (commits, resets, checkouts).

Question	Answer	Key Concept
Q34	B	Three reset modes: --soft (undo commit, keep staged), --mixed (undo commit, unstage changes, keep in working dir), --hard (undo commit, DISCARD all changes permanently).
Q35	B	Git hooks are scripts in .
Q36	B	"Squash and Merge" collapses 20 messy WIP commits ("fix typo", "debugging", "finally works") from a feature branch into ONE clean commit on main.
Q37	B	git blame shows every line of a file annotated with: commit hash, author name, date, and line number.
Q38	B	SemVer (v2.

SECTION 6: Programming Ecosystems

Question	Answer	Key Concept
Q39	B	GC (Java, Go, Python, C#) automatically reclaims memory of unreachable objects, preventing memory leaks.
Q40	B	Reflection allows runtime inspection of classes, methods, and fields without knowing them at compile time.
Q41	B	Concurrency: structure — managing multiple tasks that may overlap in time (a waiter serving many tables).
Q42	B	A monad wraps a value with context and provides flatMap (or bind) for chaining operations.
Q43	A	Compiled: source -> machine code (C, Go, Rust).
Q44	B	CPython's GIL prevents multiple threads from executing Python bytecode simultaneously — so CPU-bound multi-threaded Python code doesn't benefit from multiple cores.
Q45	B	Tree shaking statically analyzes ES6 import/export statements to identify and remove unused code.